

Sprint 4.- René Balderrama A.

Nivel 1: Entorno e ingesta híbrida (Code-First)

Escenario

El cuadro de mando financiero tarda demasiado en cargar y el coste es inaceptablemente alto. Se ha identificado que el problema crítico son los informes que cruzan ventas con regiones geográficas.

Ejercicio 1: Consulta sobre una Tabla No Optimizada (Diagnóstico)

El Country Manager de Alemania necesita revisar urgentemente las transacciones del día 12 de marzo de 2022.

Tarea:

Escribe la consulta que une (JOIN) las transacciones y las compañías.

Filtra los resultados por la fecha indicada y el país "Germany".

Sin ejecutar la consulta, realiza un "Dry Run" (auditoría de costes).

 Observación: Fíjate en que BigQuery lee prácticamente toda la tabla, aunque solo se solicite un día (Full Table Scan).

El dry run me muestra 0B a procesar debido a que es una tabla externa.

```
1 SELECT t.*, c.company_name, c.country
2 FROM
3   `sprint3-analytics-reneb.sprint3-bronze.transactions_raw` AS t
4 JOIN
5   `sprint3-analytics-reneb.sprint3-bronze.companies_raw` AS c
6   ON t.business_id = c.company_id
7 WHERE DATE(t.timestamp) = '2022-03-12' AND c.country = 'Germany';
8
```

 Esta consulta procesará 0 B cuando se ejecute.

Al ejecutarse los bytes facturados son 20MB:

```
1 SELECT t.*, c.company_name, c.country
2 FROM
3 | `sprint3-analytics-reneb.sprint3-bronze.transactions_raw` AS t
4 JOIN
5 | `sprint3-analytics-reneb.sprint3-bronze.companies_raw` AS c
6 | ON t.business_id = c.company_id
7 WHERE DATE(t.timestamp) = '2022-03-12' AND c.country = 'Germany';
8
```

✓ Se completó la consulta

> Resultados de la consulta [Chat](#) [View all jobs](#) [Guardar los resultados](#)

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución
Duración		1 s 925 ms		
Bytes procesados		12.62 MB		
Bytes facturados		20 MB		
Milisegundos de ranura		2196		
Prioridad del trabajo		INTERACTIVE		
Usar SQL heredado		false		

Ejercicio 2: Re-arquitectura y Optimización del Almacenamiento (Partition & Cluster)

Escenario:

Después del diagnóstico del ejercicio anterior, hemos confirmado que filtrar la tabla de transacciones por fecha o por empresa provoca un Full Table Scan (lectura completa de la tabla), lo que resulta ineficiente. Para solucionarlo en la capa de consumo (Gold), materializaremos una versión optimizada de la tabla de hechos.

Tarea:

Paso 1: Generación de Datos Recientes (Mocking Data)

Crea una tabla intermedia llamada `sprint3_silver.transactions_recent` a partir de la tabla `sprint3_silver.transactions_clean`. Tu objetivo es mantener todas las columnas, pero sustituir el timestamp original por uno nuevo, generado aleatoriamente para que se encuentre dentro de los últimos 50 días.

Pista técnica: Para hacerlo sin tener que escribir todas las columnas manualmente, puedes utilizar `SELECT * EXCEPT(timestamp)`. Para calcular la fecha aleatoria, tendrás que restar una cantidad de días al instante actual. Te serán muy útiles las funciones `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()` y la combinación `CAST(RAND() * 50 AS INT64)` para generar los días de desfase.

```

1 CREATE OR REPLACE TABLE `sprint3-analytics-reneb.sprint3_silver.transactions_recent`
2 AS
3 SELECT
4     * EXCEPT (timestamp),
5     TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST(RAND() * 50 AS INT64) DAY)
6     AS timestamp

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

> Resultados de la consulta [Chat](#) [View all jobs](#) [Guardar los resultados](#)

Información del trabajo	Resultados	Detalles de la ejecución	Gráfico de ejecución
Duración	1 s 659 ms		
Bytes procesados	11.09 MB		
Bytes facturados	12 MB		
Milisegundos de ranura	1742		

sprint3-analytics-reneb / Datasets / sprint3_silver / Tables / transactions_recent

☆ transactions_r... [Consulta](#) [Chat](#) [Abrir en](#) [Compartir](#) [Copiar](#) [Actualizar](#)

Esquema	Detalles	Vista previa	Estadísticas	Linaje	Perfil de datos	Calidad de los datos
Fila	company_id	declined	user_id	product_ids	timestamp	
5	b-2398	0	1	64 92 76	2026-08-19 15:10:13.457130 UTC	
6	b-2390	0	1	81 94 65	2026-09-03 15:10:13.457130 UTC	
7	b-2494	0	1	26 20 79	2026-09-12 15:10:13.457130 UTC	
8	b-2294	0	1	95	2026-09-26 15:10:13.457130 UTC	

Paso 2: Creación de la Tabla Optimizada (Partitioning & Clustering)

Ahora, crea tu tabla definitiva `sprint3_gold.fact_transactions_optimized` a partir de los datos recientes que acabas de generar en `transactions_recent`.

Debes construir la sentencia DDL para configurar la tabla con las siguientes estrategias físicas de almacenamiento:

Particionamiento (Partitioning): Divide la tabla por la fecha del campo `DATE(timestamp)`. Esto permitirá que las consultas que filtren por día (`WHERE date = ...`) solo lean la partición necesaria.

Clustering: Ordena los datos dentro de cada partición por `business_id`. Esto acelerará drásticamente los filtros por compañía y los cruces (`JOINS`) con la dimensión de empresas.

```

1 CREATE OR REPLACE TABLE `sprint3-analytics-reneb.sprint3_gold.fact_transactions_optimized`
2 PARTITION BY DATE(timestamp) CLUSTER BY company_id
3 AS
4 SELECT * FROM `sprint3-analytics-reneb.sprint3_silver.transactions_recent`;
5
6

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

> Resultados de la consulta [Chat](#) [View all jobs](#) [Guardar los resultados](#)

Información del trabajo	Resultados	Detalles de la ejecución	Gráfico de ejecución
Duración	2 s 711 ms		
Bytes procesados	11.85 MB		
Bytes facturados	12 MB		
Milisegundos de ranura	17550		

☆ fact_transacti... [Consulta](#) [Chat](#) [Abrir en](#)

ⓘ Esta es una tabla particionada. [Learn more](#)

Esquema	Detalles	Vista previa	Estadísticas	Linaje	Perf
Tipo de tabla	Particionada				
Particionada por	DAY				
Particionada en el campo	timestamp				
Vencimiento de la partición	60 días				
Filtro de partición	No es obligatorio				
Agrupada por	company_id				

Ejercicio 3: La Prueba del Algodón (Benchmark)

Escenario:

Ya has creado la tabla `sprint3_gold.fact_transactions_optimized` con las estrategias de Particionamiento y Clustering. Ahora toca demostrar al equipo la mejora de rendimiento y el ahorro de costes que has conseguido.

Simularemos una petición típica de negocio:

"Queremos ver todas las transacciones de los últimos 30 días".

🧩 Tarea:

El objetivo es claro y directo: aplicar exactamente la misma consulta a dos tablas diferentes y comparar su coste computacional.

Construye una consulta SQL que seleccione todas las columnas (SELECT *) y filtre los datos de los últimos 30 días.

⚠️ NO ejecutes las consultas todavía. En este ejercicio solo utilizaremos el validador de BigQuery ("Dry Run", arriba a la derecha del editor) para visualizar los Bytes procesados de forma gratuita.

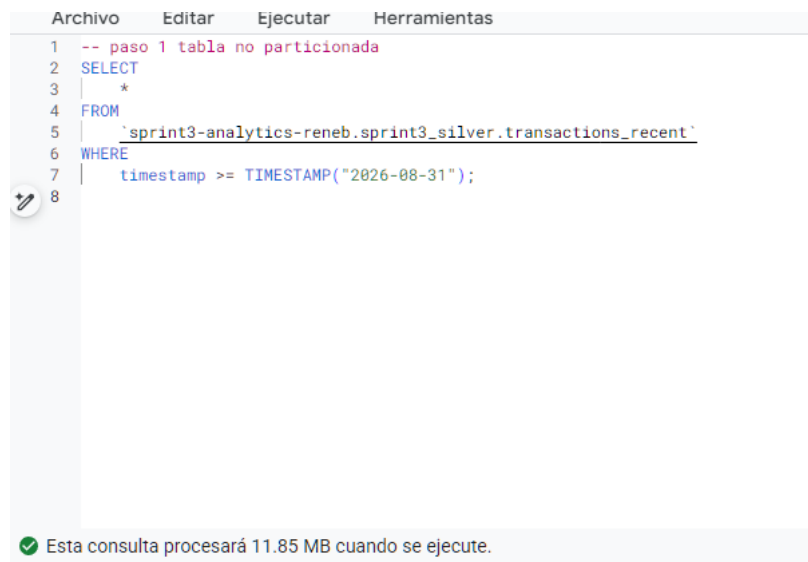
Realiza la comparativa siguiendo estos dos pasos:

Paso 1 — Tabla no optimizada

Apunta tu consulta a la tabla:

sprint3_silver.transactions_recent

Observa el validador y haz una captura de pantalla de los bytes que procesaría.



```
Archivo  Editar  Ejecutar  Herramientas
1  -- paso 1 tabla no particionada
2  SELECT
3  |
4  | *
5  FROM
6  | `sprint3-analytics-reneb.sprint3_silver.transactions_recent`
7  WHERE
8  | timestamp >= TIMESTAMP("2026-08-31");
```

Esta consulta procesará 11.85 MB cuando se ejecute.

Paso 2 — Tabla optimizada

Sin cambiar nada de la lógica de tu consulta, modifica únicamente el FROM para que ahora apunte a la tabla particionada:

sprint3_gold.fact_transactions_optimized

Observa cómo cambia el validador y haz la segunda captura de pantalla.

```
1 -- paso 2 tabla particionada
2 SELECT
3   *
4 FROM
5   `sprint3-analytics-reneb.sprint3_gold.fact_transactions_optimized`
6 WHERE
7   timestamp >= TIMESTAMP("2026-08-31");
8 |
```

✓ Esta consulta procesará 7.25 MB cuando se ejecute.

Al implementar una arquitectura de almacenamiento optimizada mediante particionamiento por fecha, se ha logrado reducir el procesamiento de datos de 12,43 MB a solo 7,60 MB para una consulta de los últimos 30 días. Esto se traduce en un ahorro directo del 38,85% en costes y una mejora significativa en la velocidad de carga, ya que BigQuery ahora solo lee las particiones relevantes en lugar de realizar un escaneo completo de la tabla, demostrando que la solución es técnica y financieramente mucho más eficiente.

Ejercicio 4: Smart Caching (Vistas Materializadas)

Escenario:

El director general tiene un cuadro de mando que muestra las "Ventas Totales por Día". Él (y 50 managers más) actualizan este gráfico constantemente.

Problema:

Si utilizas una consulta normal o una Vista estándar, cada vez que alguien actualiza la página, BigQuery recalcula la suma de millones de filas. Es lento y estás pagando por el mismo cálculo una y otra vez.

Solución:

Utilizar una Vista Materializada. Esta tecnología guarda el resultado del cálculo en la caché y solo lo actualiza si entran datos nuevos (de forma incremental).

🧩 Tarea:

Crea una Vista Materializada llamada `sprint3\_gold.mv\_daily\_sales` que muestre las Ventas Totales por Día.

👉 Entrega:

Código para crear la vista materializada.

```
1 CREATE OR REPLACE MATERIALIZED VIEW `sprint3-analytics-reneb.sprint3_gold.mv_daily_sales`
2 AS
3 SELECT
4     DATE(timestamp) AS sales_date,
5     SUM(amount) AS total_sales,
6     COUNT(*) AS total_transactions
7 FROM
8     `sprint3-analytics-reneb.sprint3_gold.fact_transactions_optimized`
9 WHERE declined = 0
10 GROUP BY
11     1;
12
```

✓ Esta consulta procesará 0 B cuando se ejecute.

Realiza una consulta que muestre la vista creada y haz una captura de pantalla que muestre en su conjunto la vista materializada creada, junto con la consulta ejecutada y los bytes procesados.

sprint3-analytics-reneb / Datasets / sprint3_gold / Tables / mv_daily_sales

☆ mv_daily_sales 🔍 Consulta 🗨 Chat 📄 Abrir en 👤 Compartir 🗑 Borrar

Esquema Detalles Estadísticas Linaje Perfil de datos Calidad de los datos

🔍 Filtro Escribir el nombre o valor de la propiedad

☐	Nombre del campo	Tipo	Modo	Descripción	Clave	Intercalación	Valor predete
☐	sales_date	DATE	NULLABLE	-	-	-	-
☐	total_sales	FLOAT	NULLABLE	-	-	-	-
☐	total_transactions	INTEGER	NULLABLE	-	-	-	-

Archivo

Editar

Ejecutar

Herramientas

```
1 SELECT * FROM `sprint3-analytics-reneb.sprint3_gold.mv_daily_sales`
2 ORDER BY sales_date DESC
3 ;
```

Esta consulta procesará 1.2 KB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

>

Resultados de la consulta

Chat

View all jobs

Guardar los resultados

Abrir en

Información del trabajo

Resultados

Visualización

JSON

Detalles de la ejecución

Gráfico de ejecución

Fila	sales_date	total_sales	total_transactions	
1	2026-09-30	269150.8800000...	1053	
2	2026-09-29	508983.4000000...	1989	
3	2026-09-28	499777.5000000...	1942	
4	2026-09-27	511343.1400000...	1977	
5	2026-09-26	534729.8400000...	2059	

Resultados por página:

50

1 – 50 de 51

Nivel 2: SQL Analítico Avanzado

Escenario:

La dirección empieza a hacer preguntas que no se resuelven con un simple `GROUP BY`. Necesitamos análisis de tendencias y comportamiento de usuario.

Ejercicio 1: Perfilado de Clientes VIP (Métricas Agregadas con CTEs)

Escenario:

El equipo de Marketing quiere analizar el comportamiento de nuestros mejores clientes para diseñar la estrategia del próximo año. Definimos "VIP" como aquellos con un gasto acumulado superior a 500€. Necesitan un informe que, para cada VIP, muestre su nombre, contacto y su patrón de compra: cuántas veces ha comprado, cuánto gasta de media y cuál fue su compra récord.

Tarea:

1. Crea una CTE llamada `VIP\_Stats` que agrupe por usuario y calcule:

El Gasto Total (`SUM`).

La Cantidad de Transacciones (`COUNT`).

El Ticket Medio (`AVG`), redondeado a 2 decimales.

La Compra Máxima (`MAX`).

Filtro: Mantén solo aquellos cuyo Gasto Total sea > 500.

```

1 WITH VIP_Stats AS (
2     SELECT
3         user_id, ROUND(SUM(amount),2) AS total_gastado, COUNT(transaction_id) AS num_compras, ROUND(AVG(amount), 2) AS
4         tiquet_media,
5         MAX(amount) AS max_compra
6     FROM `sprint3-analytics-reneb.sprint3_gold.fact_transactions_optimized`
7     WHERE declined = 0
8     GROUP BY user_id
9     HAVING total_gastado >= 500 )
10
11 SELECT v.user_id, CONCAT(u.name, ' ', u.surname) AS nom_complet, u.email, v.num_compras, v.tiquet_media,
12 v.max_compra, v.total_gastado
13 FROM VIP_Stats AS v
14 JOIN `sprint3-analytics-reneb.sprint3_silver.users_combined` AS u
15 ON v.user_id = u.user_id
16 ORDER BY v.total_gastado DESC;
17

```

Se completó la consulta

2. Cruza la CTE con `users\_combined` para obtener los datos personales.

Requisitos de Salida:

Columnas: `user\_id`, `nom\_complet`, `email`, `num\_compres`, `tiquet\_mig`,
`max\_compra`, `total\_gastat`.

Orden: Ordenado por `total\_gastat` descendente.

Se completó la consulta

Resultados de la consulta

Chat

View all jobs

Guardar los resultados

Abrir en

Información del trabajo

Resultados

Visualización

JSON

Detalles de la ejecución

Gráfico de ejecución

Fila	user_id	nom_complet	email	num_compras	tiquet_media	max_compra	total_gastado
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.39
3	455	Fwll Phylgnmhnd	fwll.phylgnmhnd@example.com	66	522.59	724.44	34491.27
4	417	Fqatil Rdwcjvoa	fqatil.rdwcjvoa@example.com	73	421.87	675.78	30796.3
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.79
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	78	304.0	604.9	23711.9
9	349	Yxxcjk Fnceqbkxmt	yxxcjk.fnceqbkxmt@example.c...	53	443.48	675.05	23504.47
10	317	Qikcvc Nvfoifqdh	qikcvc.nvfoifqdh@example.com	54	351.69	600.77	18991.45
11	388	Rbjupnac Cfjykd	rbjupnac.cfjykd@example.com	62	297.3	479.03	18432.84

Resultados por página: 501 - 50 de 5000

Ejercicio 2: Análisis de Tendencias (Window Functions sobre Vistas)

Escenario:

La Dirección Financiera quiere monitorizar la "velocidad de ventas" diaria. Necesitan un informe que compare el rendimiento de cada día contra el día anterior para detectar caídas bruscas o picos de crecimiento (Day-over-Day Growth).

Estrategia de Optimización:

Para este cálculo NO debes ir a la tabla de hechos original ni recalculas sumas. Como buen Data Engineer, debes utilizar la Vista Materializada sprint3_gold.mv_daily_sales que creaste en el ejercicio anterior. Esto garantiza que el análisis de tendencias sea instantáneo y no consuma recursos de cómputo innecesarios.

Tarea:

Crea una consulta sobre la vista materializada que genere un informe con las siguientes 4 columnas:

1. Data: La fecha de venta.
2. Vendes_Avui: El total vendido ese día.
3. Vendes_Ahir: El total vendido el día anterior (usando funciones de ventana).
4. Diff_Percentual: El porcentaje de crecimiento o decrecimiento respecto al día anterior, redondeado a 2 decimales.

Pista Técnica:

Utiliza la función LAG(campo) OVER (ORDER BY fecha) para leer el valor de la fila anterior sin necesidad de hacer JOINS complejos.

```
1 WITH
2   sales_lag AS (
3     SELECT
4       sales_date AS Data,
5       total_sales AS Vendes_Avui,
6       LAG(total_sales) OVER (ORDER BY sales_date) AS Vendes_Ahir
7     FROM `sprint3-analytics-reneb.sprint3_gold.mv_daily_sales`
8   )
9 SELECT
10  Data,
11  ROUND(Vendes_Avui, 2) AS Vendes_Avui,
12  ROUND(Vendes_Ahir, 2) AS Vendes_Ahir,
13  ROUND(SAFE_DIVIDE(Vendes_Avui - Vendes_Ahir, Vendes_Ahir) * 100, 2)
14  AS Diff_Percentual
15 FROM sales_lag
16 ORDER BY Data DESC;
```

✓ Esta consulta procesará 816 B cuando se ejecute.

Resultado consulta:

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	Data	Vendes_Avui	Vendes_Ahnr	Diff_Percentual		
1	2026-09-30	269150.88	508983.4	-47.12		
2	2026-09-29	508983.4	499777.5	1.84		
3	2026-09-28	499777.5	511343.14	-2.26		
4	2026-09-27	511343.14	534729.84	-4.37		
5	2026-09-26	534729.84	534674.15	0.01		
6	2026-09-25	534674.15	539500.06	-0.89		
7	2026-09-24	539500.06	502463.1	7.37		
8	2026-09-23	502463.1	527092.68	-4.67		
9	2026-09-22	527092.68	503503.32	4.69		
10	2026-09-21	503503.32	524123.23	-3.93		
11	2026-09-20	524123.23	504981.84	3.79		
12	2026-09-19	504981.84	527266.84	-4.23		

Resultados por página: 50 1 - 50 de 51

Ejercicio 3: Totales Acumulados (Running Totals sobre Vistas)

Escenario:

El director Financiero (CFO) necesita visualizar la curva de crecimiento anual. No le interesa tanto lo que se vendió hoy, sino cuánto llevamos facturado en total desde el inicio del año hasta hoy (YTD - Year To Date).

Tarea:

Utilizando la vista materializada mv_daily_sales (para no recalculer agregaciones base), genera un informe con tres columnas:

1. Data: La fecha.
2. Vendes del Dia: Ventas del día, redondeado a 2 decimales.
3. Vendes Acumulades YTD: Una suma progresiva de las ventas que debe reiniciarse cada 1 de enero. El resultado final debe estar redondeado a 2 decimales.

Pista Técnica:

Para conseguir el reinicio anual, necesitas dividir la ventana usando PARTITION BY EXTRACT(YEAR FROM fecha). La estructura completa sería: SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Archivo	Editar	Ejecutar	Herramientas
<pre> 1 SELECT 2 sales_date AS Data, 3 ROUND(CAST(total_sales AS NUMERIC), 2) AS Vendes_del_Dia, 4 ROUND(SUM(CAST(total_sales AS NUMERIC)) 5 OVER (PARTITION BY EXTRACT(YEAR FROM sales_date) 6 ORDER BY sales_date 7 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 2) AS Vendes_Acumulades_YTD 8 FROM `sprint3-analytics-reneb.sprint3_gold.my_daily_sales` 9 ORDER BY Data DESC; </pre>			
Esta consulta procesará 816 B cuando se ejecute. Se está usando la cuota de procesamiento según demanda			
> Resultados de la consulta Chat View all jobs Guardar los resultados			
Información del trabajo Resultados Visualización JSON Detalles de la ejecución Gráfico de ejecución			
Fila	Data	Vendes_del_Dia	Vendes_Acumula...
1	2026-09-30	269150.88	25830384.42
2	2026-09-29	508983.4	25561233.54
3	2026-09-28	499777.5	25052250.14
4	2026-09-27	511343.14	24552472.64
5	2026-09-26	534729.84	24041129.5
Resultados por página: 50 1 - 50 de 51			

Ejercicio 4: Fidelización y Valor del Cliente (Filtrado Avanzado)

Escenario:

El equipo de Marketing lanza la campaña "A la tercera va la vencida". Quieren enviar un regalo a los usuarios que hayan completado su tercera transacción. Sin embargo, el presupuesto del regalo no es fijo: depende de la "calidad" del cliente en sus inicios. Necesitan saber el Ticket Medio de estas 3 primeras compras para segmentar si envían un detalle básico o un regalo *premium*.

Tarea:

Genera un listado de los usuarios que han alcanzado (o superado) su tercera compra, mostrando:

- 1. Datos de usuario: (`user\_id`, `nom\_complet`, `email`).
- 2. La fecha y el importe exacto de la 3ª compra.
- 3. Media de las 3 primeras: La media de gasto de sus transacciones 1, 2 y 3.

Pista Técnica:

Debes utilizar la función `ROW\_NUMBER()` para establecer el orden cronológico de las compras.

Debes utilizar la cláusula `QUALIFY` para filtrar.

Reto de Optimización: Busca la estrategia de menor coste computacional. Ten en cuenta que calcular medias sobre todo el histórico es costoso. Intenta filtrar las filas necesarias antes de realizar la agregación final.

```
1 WITH
2   RankedPurchases AS (
3     SELECT user_id, timestamp AS fecha_3a_compra, amount AS importe_3a_compra,
4           ROW_NUMBER()
5             OVER (PARTITION BY user_id ORDER BY timestamp ASC) AS purchase_rank,
6           AVG(amount)
7             OVER (
8               PARTITION BY user_id
9                 ORDER BY timestamp ASC
10                ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS media_3_primeras
11   FROM `sprint3-analytics-reneb.sprint3_gold.fact_transactions_optimized`
12  WHERE declined = 0
13  QUALIFY purchase_rank = 3 )
14 SELECT
15   r.user_id, CONCAT(u.name, ' ', u.surname) AS nom_complet, u.email, r.fecha_3a_compra,
16   ROUND(CAST(r.importe_3a_compra AS NUMERIC), 2) AS importe_3a_compra,
17   ROUND(CAST(r.media_3_primeras AS NUMERIC), 2) AS media_3_primeras
18 FROM RankedPurchases r
19 JOIN `sprint3-analytics-reneb.sprint3_silver.users_combined` u
20   ON r.user_id = u.user_id
21 ORDER BY media_3_primeras DESC;
22
```

Se completó la consulta

Se completó la consulta

Resultados de la consulta

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución	
Fila	user_id	nom_complet	email	fecha_3a_compra	importe_3a_com...	media_3_primeras	
1	4704	Cdkmoo Gaqthofe	cdkmoo.gaqthofe@example.com	2026-08-20 15:10:13.457130 UTC	437.8	591.96	
2	638	Gbkdiv Mgbjwva	gbkdiv.mgbjwva@example.com	2026-08-18 15:10:13.457130 UTC	434.56	579.46	
3	180	Amelia Valenzuela	ac.nulla@google.couk	2026-08-15 15:10:13.457130 UTC	631.57	565.96	
4	455	Fwfl Phylgnmhnd	fwfl.phylgnmhnd@example.com	2026-08-14 15:10:13.457130 UTC	678.61	541.91	
5	1182	Rutkmp Jylvzugf	rutkmp.jylvzugf@example.com	2026-08-19 15:10:13.457130 UTC	556.96	541.82	
6	2521	Catdaw Skdxtfep	catdaw.skdxtfep@example.com	2026-08-18 15:10:13.457130 UTC	641.64	530.85	
7	213	Blake Strickland	ac.libero.nec@yahoo.com	2026-08-13 15:10:13.457130 UTC	704.19	528.81	
8	29	Deacon Sharpe	hendrerit@icloud.net	2026-08-18 15:10:13.457130 UTC	557.06	524.85	
9	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	2026-08-12 15:10:13.457130 UTC	524.84	524.84	
10	3417	Sxeoji Zxxczpf	sxeoji.zxxczpf@example.com	2026-08-13 15:10:13.457130 UTC	576.62	523.39	
11	2376	Pqwfvm Smdkjzvg	pqwfvm.smdkjzvg@example.co...	2026-08-20 15:10:13.457130 UTC	303.97	521.36	

Resultados por página: 50 1 - 50 de 5000

Nivel 3: Analytics Engineering (Arrays y Automatización)

Escenario:

Marketing necesita urgentemente un Ranking de Productos Más Vendidos.

Ejercicio 1: Desanidamiento y Aplanamiento de Datos (Unnesting)

Escenario:

Actualmente, en la tabla de transacciones, los productos vendidos están "ocultos" dentro de un Array (una lista de IDs). El equipo de ventas quiere analizar qué productos específicos se venden más, pero no pueden hacerlo si los datos están comprimidos en una lista. Necesitan una Tabla Detallada (Flat Table) donde cada fila

represente un producto vendido, aunque eso signifique que el ID de la transacción se repita.

Tarea:

Crea la tabla `dim\_transactions\_flat` desnormalizando la información. Debes "explotar" el array de productos y cruzarlo con el catálogo maestro para obtener los nombres y precios individuales.

Pasos Técnicos:

1. Utiliza `CROSS JOIN UNNEST(product\_ids)` para transformar el Array en filas individuales.
2. Realiza un `JOIN` con la tabla `products` para obtener el nombre (`name`) y precio (`price`) de cada ítem.

Nota: Ten cuidado con los tipos de datos al realizar el `JOIN` (Array de INT64 vs Product_ID STRING).

3. No agrupes el resultado. Queremos ver el desglose línea por línea. – faltaron campos de la tabla transactions_clean

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-reneb.sprint3_gold.dim_transactions_flat` AS
2 SELECT t.transaction_id, t.timestamp, t.user_id, p.product_id, t.amount AS total_ticket_amount, t.declined, p.name AS
   product_name,
3 p.price AS individual_price, t.card_id, t.company_id
4
5 FROM `sprint3-analytics-reneb.sprint3_silver.transactions_clean` AS t
6
7 CROSS JOIN UNNEST(t.product_ids) AS individual_product_id
8
9 JOIN `sprint3-analytics-reneb.sprint3_silver.products_clean` AS p
10 ON individual_product_id = p.product_id ;
11
```

Se completó la consulta

sprint3-analytics-reneb / Datasets / sprint3_gold / Tables / dim_transactions_flat

dim_transacti... Consulta Chat Abrir en Compartir Copiar Instantánea Borrar Exportar o sincronizar Actualizar

Esquema	Detalles	Vista previa	Estadísticas	Linaje	Perfil de datos	Calidad de los datos			
Fila	transaction_id	timestamp	user_id	product_id	total_ticket_amount	declined	product_name	individual_price	card_id
1	777DD5BA-F10D-49DA-870F-BD...	2021-07-22 08:54:16 UTC	183	1	161.11	0	Direwolf Stannis	161.11	CcU-3582
2	37560F5B-590E-4A3E-B869-8D...	2021-07-26 19:18:41 UTC	183	1	161.11	0	Direwolf Stannis	161.11	CcU-3582
3	A9C23899-B1A2-447E-A009-9D...	2021-07-08 17:37:02 UTC	183	1	161.11	0	Direwolf Stannis	161.11	CcU-3582
4	2E7D2A34-ED52-4E60-A395-A1...	2024-11-05 19:37:36 UTC	4084	1	611.51	0	Direwolf Stannis	161.11	CcS-8665
5	2492B160-509D-44FD-B172-7D...	2021-07-06 05:03:57 UTC	183	1	161.11	0	Direwolf Stannis	161.11	CcU-3582
6	6C7985FC-2038-477D-ABE6-85...	2021-07-01 08:30:28 UTC	183	1	161.11	0	Direwolf Stannis	161.11	CcU-3582
7	CO05E2CE-DF82-4F1F-9D7D-00...	2022-09-28 22:38:38 UTC	1953	1	499.51	0	Direwolf Stannis	161.11	CcS-6534
8	61B9D45E-C5E8-42FA-A6AF-F5...	2021-07-23 18:48:38 UTC	183	1	161.11	0	Direwolf Stannis	161.11	CcU-3582
9	8A52CD9E-1E1E-43CF-A038-01...	2022-12-30 20:06:57 UTC	2282	1	396.05	0	Direwolf Stannis	161.11	CcS-6863
10	78187BAB-D334-466E-816C-D9...	2021-02-06 20:27:16 UTC	2558	1	298.92	0	Direwolf Stannis	161.11	CcS-7139
11	33131252-8E5B-4D17-84FC-48...	2021-07-13 00:24:44 UTC	183	1	161.11	0	Direwolf Stannis	161.11	CcU-3582
12	FOC96452-7781-41CF-BE26-0D...	2021-07-29 22:07:02 UTC	183	1	161.11	0	Direwolf Stannis	161.11	CcU-3582
13	1CCBA9B9-F70D-4FBE-8641-21...	2016-12-27 01:53:37 UTC	4520	1	684.22	0	Direwolf Stannis	161.11	CcS-9101
14	647H98F8-6077-49E1-A1B0-8D...	2021-07-15 11:47:13 UTC	183	1	161.11	0	Direwolf Stannis	161.11	CcU-3582

Ejercicio 2: El Ranking de Ventas (Agregación Simple

Escenario:

Ahora que has preparado la infraestructura y dispones de la tabla `dim\_transactions\_flat` (donde cada fila equivale a un producto vendido), generar informes analíticos es trivial. Ya no necesitas funciones complejas de desanidamiento para responder preguntas básicas de negocio.

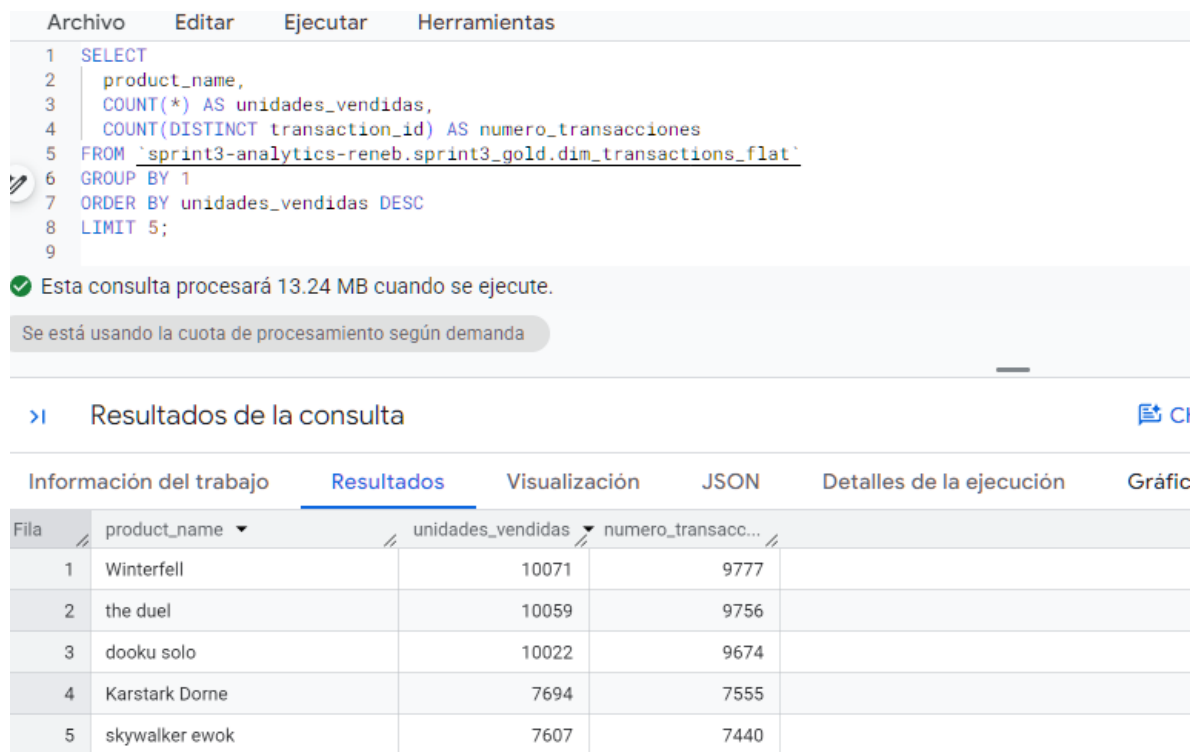
Tarea:

Genera el Top 5 de productos más vendidos en la historia de la compañía.

Método:

Consulta directamente tu nueva tabla desnormalizada (`dim\_transactions\_flat`).

Al ser la tabla plana, simplemente necesitas una agrupación estándar (`GROUP BY`) por el nombre del producto y un conteo (`COUNT`) de las filas.



```
1 SELECT
2   product_name,
3   COUNT(*) AS unidades_vendidas,
4   COUNT(DISTINCT transaction_id) AS numero_transacciones
5 FROM `sprint3-analytics-reneb.sprint3_gold.dim_transactions_flat`
6 GROUP BY 1
7 ORDER BY unidades_vendidas DESC
8 LIMIT 5;
9
```

✓ Esta consulta procesará 13.24 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

> Resultados de la consulta

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico
Fila	product_name	unidades_vendidas	numero_transacc...			
1	Winterfell	10071	9777			
2	the duel	10059	9756			
3	dooku solo	10022	9674			
4	Karstark Dorne	7694	7555			
5	skywalker ewok	7607	7440			

Ejercicio 3: Automatización del Pipeline y Visualización

Escenario:

El informe estático del ejercicio anterior ha gustado mucho, pero el Director de Marketing necesita profesionalizarlo para la toma de decisiones diaria. Tiene dos requisitos urgentes:

1. Lógica Centralizada: Quieren ver el precio con IVA (21%) desglosado. Sin embargo, como el impuesto puede cambiar según el país o el año, te prohíben poner el cálculo ``precio * 1,21`` directamente en el código ("Hardcoded"). Debes utilizar una función reutilizable.

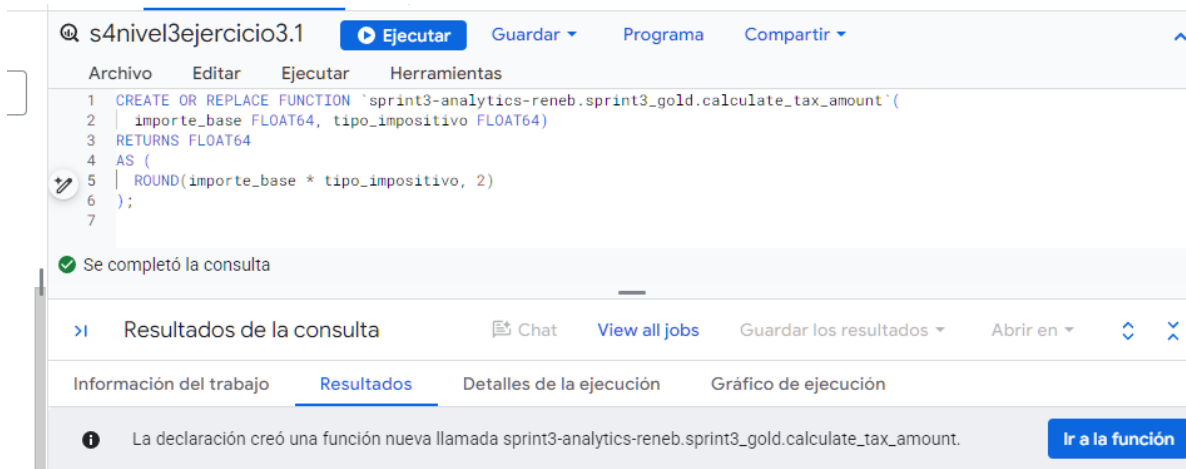
2. Frescura de los Datos: El panel debe estar actualizado automáticamente cada mañana a las 07:00 AM, antes de la reunión diaria de ventas.

Tarea:

Debes evolucionar tu tabla ``dim_transactions_flat`` para incluir el cálculo de impuestos y automatizar su regeneración.

Pasos Técnicos:

1. User Defined Functions (UDF): Crea una función SQL persistente llamada ``calculate_tax(amount)`` que reciba un valor numérico y devuelva el resultado aplicando el 21% de impuesto.



2. Integración y Orquestación:

Modifica el código de creación de la tabla (del Ejercicio 1) para que utilice tu nueva función ``calculate_tax`` y genere una nueva columna: ``product_price_tax_inc``.

Configura una Scheduled Query en BigQuery para que esta sentencia (CTAS) se ejecute automáticamente cada día a las 07:00 AM.

El código SQL actualizado de la creación de la tabla.

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-reneb.sprint3_gold.dim_transactions_flat`
2 AS
3 SELECT
4   transaction_id, timestamp, user_id, product_id, total_ticket_amount, declined, product_name,
5   individual_price, card_id, company_id,
6   ROUND(
7     individual_price + `sprint3-analytics-reneb.sprint3_gold.calculate_tax_amount`(
8       individual_price, 0.21),
9     2)
10   AS product_price_tax_inc
11 FROM `sprint3-analytics-reneb.sprint3_gold.dim_transactions_flat`;
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Una captura de pantalla demostrando que la Scheduled Query está programada.

Exportar como consulta programada

✓ Configuración válida.

Detalles y programación

Nombre de la consulta programada *
Daily Refresh dim_transactions_flat

Opciones de programación

Frecuencia de repetición *
Horas

Frecuencia de repetición *
24 horas ?

☐ Comenzar ahora ☒ Comenzar a la hora definida

Fecha de inicio y hora de ejecución *
4/10/26, 7:00 a.m. 0000

☒ Sin hora de finalización programada
☐ Programar una hora de finalización

Guardando... Cancelar

3. Visualización (BI): Conecta Looker Studio a tu tabla `dim\_transactions\_flat` y crea un Dashboard: "Monitor de Rendimiento de Ventas".

Entrega:

Un enlace al gráfico en Looker Studio.

<https://datastudio.google.com/reporting/6d5bae1f-98af-4b1f-8605-7260cfef7882>

Propuesta de Dashboard:

"Monitor de Rendimiento de Ventas"

Configura una página con 3 secciones clave:

1. Los "Big Numbers" (KPIs Generales):

En la parte superior, coloca tres tarjetas de resultados (Scorecards) para proporcionar contexto inmediato.

KPI 1: Ingresos Totales (Con IVA)

Métrica: ``SUM(product_price_tax_inc)``

Por qué: Muestra el volumen real de dinero que entra.

Transacciones Únicas (Pedidos)

Métrica: ``COUNT_DISTINCT(transaction_id)``

Ticket Medio por Pedido

Campo Calculado: Crea un campo en Looker: ``SUM(product_price_tax_inc) / COUNT_DISTINCT(transaction_id)``

2. Evolución Temporal (Gráfico de Series Temporales)

Debajo de los KPIs, un gráfico de líneas que ocupe todo el ancho.

Dimensión: ``timestamp`` (Desglosado por Fecha).

Métrica: ``SUM(product_price_tax_inc)``.

Línea de Referencia (Opcional): Añade una media constante o una línea de tendencia.

Valor: Permite a Marketing ver si las ventas suben o bajan día a día, o si hubo un pico debido a alguna campaña específica.

3. El "Top Productos" Enriquecido (Tabla con Mapa de Calor)

Olvidate del gráfico de barras simple. Utiliza una tabla con barras incrustadas o un mapa de calor.

Dimensión: ``product_name``.

Métrica 1: `COUNT(\*)` → Renómbrala como ****"Unidades Vendidas"**.

Métrica 2: `SUM(product\_price\_tax\_inc)` → Renómbrala como **"Ingresos Totales"**.

Métrica 3: `SUM(product\_price\_tax\_inc) - SUM(product\_unit\_price)` → Renómbrala como **"IVA Recaudado"** (Esto demuestra el valor de tu UDF).

Estilo: En la configuración de estilo de la tabla, activa la opción **"Mostrar número como: Barra"** o **"Mapa de calor"** para la columna de Ingresos.

